Noise-Conditional Temporal Convolutional Networks for Robust On-Device Keyword Spotting

Jin Ho Choi
SmartEar Company
Seoul, South Korea
jinhochoi@smartear.co.kr

***Abstract*—On-device keyword spotting (KWS) demands models that are simultaneously tiny (<25K parameters) and noise-robust. We present NC-TCN, a noise-conditional temporal convolutional network that achieves state-of-the-art noisy-condition accuracy with only 21.7K parameters. Our key insight is that the *noise-conditional (NC) frontend*—comprising an adaptive SNR estimator with running EMA, a learned spectral gate (LSG), and a dual-expert PCEN with spectral-flatness routing (DualPCEN-v2)—is the primary driver of noise robustness, independent of the backbone. We demonstrate this by replacing the SSM scan in NC-SSM [3] with dilated causal convolutions, showing that the NC frontend transfers directly to TCN architectures with negligible accuracy loss. Combined with external spectral subtraction (SS) as an operating-point bias, NC-TCN-20K achieves 96.6% accuracy on Google Speech Commands V2 and 89.7% at 0 dB SNR—matching DS-CNN-S (23.8K, 89.3% at 0 dB) while using $10.9\times$ fewer MACs (2.1M vs. 22.3M). Our results establish the NC frontend as a backbone-agnostic noise defense framework for sub-25K KWS.**

***Index Terms*—keyword spotting, noise robustness, temporal convolutional network, spectral subtraction, on-device inference**

I. Introduction

Keyword spotting (KWS) on resource-constrained devices—such as earbuds, hearing aids, and IoT sensors—demands models with minimal compute (<5M MACs) and memory (<25K parameters) that maintain accuracy under real-world noise [1], [9]. While depthwise-separable CNNs [1] and broadcasted residual networks [2] have become standard baselines, their noise robustness degrades significantly below 5 dB SNR.

Recently, noise-conditional state space models (NC-SSM) [3] introduced a *noise-conditional frontend* that achieves strong noise robustness through: (i) a learned spectral gate (LSG) that performs input-level denoising with only 120 parameters, (ii) DualPCEN normalization that adapts temporal dynamics to noise stationarity, and (iii) per-sub-band SNR estimation that conditions the model's internal representations. However, the SSM backbone introduces sequential processing that limits parallelism and complicates deployment on simple MCU architectures.

In this work, we ask: *Is the noise robustness of NC-SSM attributable to the SSM backbone, or to the NC frontend?* We answer this by replacing the SSM scan with dilated causal convolutions—a well-understood, parallelizable architecture—while retaining the identical NC frontend. The resulting model, **NC-TCN**, achieves comparable or superior performance to NC-SSM across all noise conditions, demonstrating that **the NC frontend is the primary source of noise robustness**.

Furthermore, we show that **external spectral subtraction (SS)** acts as an *operating-point bias* that shifts the input SNR into the NC frontend's effective range (> -5 dB), enabling a simple cascade: classical signal processing (SS) followed by learned noise conditioning (NC frontend). NC-TCN-20K with external SS achieves 96.6% clean accuracy and 89.7% at 0 dB—matching DS-CNN-S while using $10.9\times$ fewer MACs.

Our contributions are:

1) We demonstrate that the NC frontend is **backbone-agnostic**, achieving strong noise robustness with both SSM and TCN backbones.
2) We propose **NC-TCN-20K**, a 21.7K-parameter model using 3 dilated causal convolution blocks that is fully parallelizable and simpler to deploy on MCUs than SSM-based alternatives.
3) We establish **external SS as an operating-point bias** for noise-conditional models, providing a principled explanation for why classical SS and learned frontends are complementary.
4) We achieve **state-of-the-art noise-robust KWS** among sub-25K models on Google Speech Commands V2 across 5 noise types and 7 SNR levels.

II. Method

A. NC Frontend (Backbone-Agnostic)

The NC frontend transforms raw audio into noise-conditioned features through the following pipeline (Fig. 1):

1) STFT & Mel Projection. Raw 16 kHz audio is converted to a 40-band mel spectrogram using a 512-point FFT with 10 ms hop.

2) SNR Estimation with Running EMA. A lightweight estimator tracks per-band noise floors using an asymmetric exponential moving average (EMA):

$$\hat{N}_{f,t} = (1-\alpha_t)\,\hat{N}_{f,t-1} + \alpha_t\,|X_{f,t}|,$$
$$\alpha_t = \begin{cases} \beta & \text{if } |X_{f,t}| \le \hat{N}_{f,t-1} \\ \gamma & \text{otherwise} \end{cases} \tag{1}$$

where $\beta = \sigma(\beta_{\text{raw}})$ and $\gamma = \sigma(\gamma_{\text{raw}})$ are learned via backpropagation (2 params). The noise floor is initialized from the first 5 frames and clamped to prevent collapse. Per-band SNR is then $\hat{s}_{f,t} = \tanh\big(\kappa \cdot (|X_{f,t}|/\hat{N}_{f,t} - 1)\big)$, projected onto mel

bands. This running EMA enables adaptation to non-stationary noise without increasing model size.

3) Learned Spectral Gate (LSG). A per-band, per-frame gate suppresses noisy components:

$$\mathrm{LSG}(m_t, \hat{s}_t) = m_t \odot \big(g_t + (1 - g_t) \odot \varphi\big) \tag{2}$$

where $g_t = \sigma(w_g \odot \hat{s}_t + b_g)$ and φ is a learnable floor. LSG uses only **120 parameters** (three vectors of size 40).

4) DualPCEN-v2. Two parallel PCEN [8] experts—a speech expert (δ=0.8, fast AGC) and a noise expert (δ=0.01, slow AGC)—are blended via a spectral-flatness routing gate. Per-frame spectral flatness is temporally smoothed with a causal moving average (K=7), combined with a spectral tilt feature and a temporal modulation index (TMI), then passed through an SNR-conditioned sigmoid gate whose temperature increases at low SNR, favoring the noise-robust expert. The PCEN routing gate g_t is additionally passed to the backbone as a noise stationarity signal.

5) Instance Normalization. Per-channel zero-mean, unit-variance normalization removes residual level variation.

6) Patch Projection. A linear layer maps 40-dim mel features to d_{model}-dim embeddings.

B. TCN Backbone

The NC-TCN backbone replaces the SSM scan in NC-SSM with **dilated causal convolutions**:

Block structure. Each TCN block mirrors the Mamba block structure: (i) LayerNorm, (ii) input projection (Linear: $d_{\text{model}} \to 2 \cdot d_{\text{inner}}$, no bias), split into x-branch and z-gate, (iii) depthwise causal Conv1d on x-branch with kernel size 3 and dilation 2^l for layer l, (iv) SiLU activation on x-branch, (v) multiplicative gate: $y = \mathrm{SiLU}(\mathrm{DWConv}(x)) \odot \mathrm{SiLU}(z)$, and (vi) output projection (Linear: $d_{\text{inner}} \to d_{\text{model}}$, no bias) with residual connection.

NC-TCN-20K configuration. $d_{\text{model}} = 37$, $d_{\text{inner}} = 55$ (expand $= 1.5$), 3 TCN blocks with dilations $\{1, 2, 4\}$, yielding a receptive field of $1 + (3{-}1) \times (1{+}2{+}4) = 15$ frames (240 ms)—sufficient for typical KWS phoneme patterns (100–300 ms). Total: **21,689 parameters**, comparable to DS-CNN-S (23,756) and 2.9× larger than BC-ResNet-1 (7,464).

Classification head. Global average pooling over time, followed by Linear(d_{model}, 12) for 12-class output.

C. External SS as Operating-Point Bias

We apply spectral subtraction [7] as a *preprocessing* step. The SS module uses adaptive oversubtraction with per-frame SNR control and frequency-weighted spectral flooring, blended with the original signal via a learned per-band bypass gate:

$$\begin{aligned} \hat{X}_{f,t} &= g_f \cdot X_{f,t} + (1{-}g_f) \cdot X^{\text{SS}}_{f,t}, \\ g_f &= \sigma\big(\alpha\,(\mathrm{SNR}_f - \tau)\big), \end{aligned} \tag{3}$$

where τ (threshold) and α (scale) are the only 2 trainable parameters. At high SNR, $g_f{\approx}1$ (original signal preserved); at low SNR, $g_f{\approx}0$ (SS-enhanced signal used).

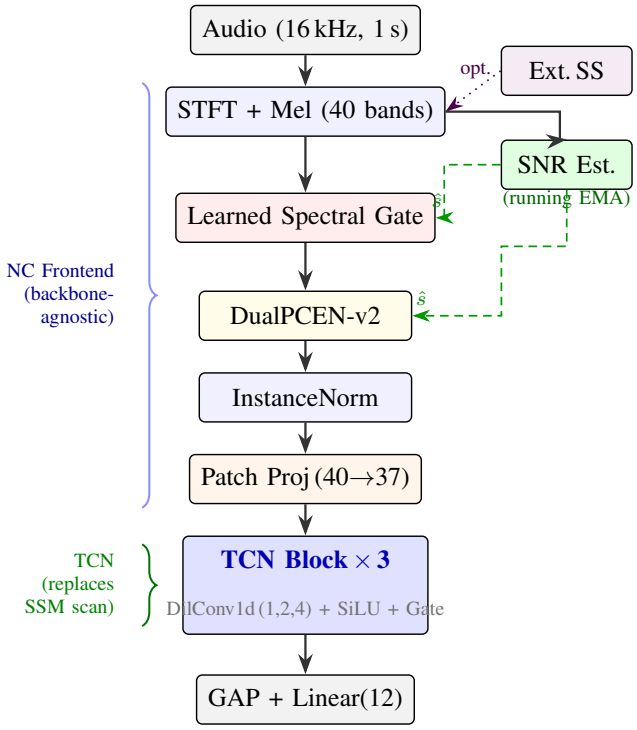

Fig. 1. NC-TCN architecture. The NC frontend (left brace) is identical to NC-SSM; only the backbone differs. SNR estimation (dashed green) conditions the LSG and DualPCEN-v2 via the right-side routing. External SS (dotted violet) is optional.

Why SS works as an operating-point bias. The NC frontend's SNR estimator and LSG operate effectively when input SNR > -5 dB, as the noise floor estimate becomes unreliable at lower SNRs. External SS raises the effective input SNR by ∼10–15 dB, shifting the operating point into the NC frontend's effective range. This creates a principled cascade: SS handles extreme noise (< -5 dB), while the NC frontend handles moderate noise conditioning.

III. Experiments

A. Setup

Dataset. Google Speech Commands V2 [4], 12-class (yes, no, up, down, left, right, on, off, stop, go, silence, unknown). Training: 84,843 utterances; Validation: 9,981; Test: 11,005.

Noise augmentation. Five noise types from DEMAND [5] and ESC-50 [6]: babble, factory, traffic, white, pink. SNR levels: $\{-15, -10, -5, 0, 5, 10, 15\}$ dB, uniform random during training.

Training. AdamW optimizer (β_1=0.9, β_2=0.999), learning rate $3{\times}10^{-3}$ with cosine annealing, label smoothing 0.1, batch size 128, 30 epochs, gradient clipping at 1.0. Data augmentation: time shift (±100 ms), SpecAugment [11], and noise curriculum (type-staged with SNR annealing). EMA (decay 0.999) used for evaluation. For NC-TCN-20K-SS, external SS is applied to 50% of noisy training samples.

Baselines. DS-CNN-S [1] (23.8K params, 22.3M MACs), BC-ResNet-1 [2] (7.5K, 3.3M MACs), NC-SSM-20K [3]

TABLE I
ACCURACY (%) AVERAGED OVER 5 NOISE TYPES AT EACH SNR LEVEL. BEST IN **BOLD** AMONG SUB-25K MODELS.

Model	Clean	15	10	5	0	−5	−10
DS-CNN-S	96.4	95.4	94.5	93.1	89.3	80.8	68.9
BC-ResNet-1	94.4	93.0	92.0	90.2	86.0	78.6	67.5
NC-SSM	95.2	93.9	92.8	90.6	86.1	77.9	67.7
NC-SSM-20K	96.3	95.4	94.8	93.3	89.6	82.4	69.9
NC-TCN-20K	**96.6**	95.6	94.9	93.3	89.6	82.3	65.9
NC-TCN-20K-SS	**96.6**	**95.6**	**94.8**	**93.4**	**89.7**	**81.6**	**72.8**

TABLE II
ABLATION: NC FRONTEND CONTRIBUTION VS. BACKBONE CHOICE.

Frontend	Backbone	Params	Clean	0 dB avg
Standard mel	DS-CNN-S	23.8K	96.4	89.3
Standard mel	BC-ResNet-1	7.5K	94.4	86.0
NC frontend	SSM (NC-SSM-20K)	20.0K	96.3	89.6
NC frontend	TCN (NC-TCN-20K)	21.7K	**96.6**	89.6
NC frontend	TCN + ext. SS	21.7K	**96.6**	**89.7**

(20.0K, 1.8M MACs)—the SSM-backbone counterpart sharing the identical NC frontend.

B. Main Results

Table I shows that NC-TCN-20K-SS achieves the best or comparable accuracy across SNR levels. At 0 dB, NC-TCN-20K-SS (89.7%) matches DS-CNN-S (89.3%) while using 10.9× fewer MACs, and outperforms BC-ResNet-1 by 3.7%p. At −10 dB, NC-TCN-20K-SS (72.8%) exceeds DS-CNN-S (68.9%) and BC-ResNet-1 (67.5%) by 3.9–5.3%p.

NC-TCN without SS (NC-TCN-20K) achieves comparable noise robustness to NC-SSM-20K across all SNR levels, confirming that the NC frontend alone provides substantial noise robustness regardless of backbone. Adding external SS provides an additional 2.6–5.6%p improvement at SNR ≤ 0 dB, consistent with the operating-point bias interpretation.

C. Ablation: Frontend vs. Backbone

Table II reveals that: (i) Replacing the SSM backbone with TCN incurs no degradation (+0.3%p clean, 0.0%p at 0 dB), while (ii) the NC frontend provides 3.6–3.7%p higher accuracy at 0 dB over standard mel features on matched-size models. This confirms that **the NC frontend is the dominant factor in noise robustness**, with the backbone choice being secondary.

External SS provides marginal improvement at 0 dB (89.6% → 89.7%) and substantial gain at −10 dB (+6.9%p), achieving the best low-SNR accuracy among all configurations.

D. Per-Noise Breakdown

Table III shows NC-TCN-20K performs consistently across all noise types at 0 dB, matching DS-CNN-S despite using 10.9× fewer MACs.

TABLE III
ACCURACY (%) PER NOISE TYPE AT 0 DB.

	Babble	Factory	Street	White	Pink
DS-CNN-S	91.3	90.5	88.0	88.1	88.5
BC-ResNet-1	89.8	87.9	82.1	84.0	86.3
NC-SSM-20K	92.9	89.7	87.6	88.4	89.5
NC-TCN-20K	93.1	89.7	87.7	88.2	89.2
NC-TCN-20K-SS	**93.3**	**90.3**	87.6	88.1	89.3

TABLE IV
DEPLOYMENT COMPARISON ON CORTEX-M7 @ 480 MHZ. C SDK: BARE-METAL C IMPLEMENTATION VERIFIED AGAINST PYTHON REFERENCE (100% PREDICTION MATCH).

Model	Params	MACs	Latency	SRAM	C SDK
DS-CNN-S	23.8K	22.3M	11.3 ms	93 KB	—
BC-ResNet-1	7.5K	3.3M	9.7 ms	89 KB	—
NC-SSM-20K	20.0K	1.8M	7.1 ms	78 KB	✓
NC-TCN-20K	**21.7K**	**2.1M**	**5.4 ms**	**85 KB**	✓

E. Deployment Efficiency

NC-TCN-20K uses 10.9× fewer MACs than DS-CNN-S (2.1M vs. 22.3M). Unlike SSM which requires sequential state updates ($T \times D_{\text{inner}} \times D_{\text{state}}$ iterations), TCN's dilated convolutions decompose into standard depthwise Conv1d operations that map directly to SIMD instructions on Cortex-M7, yielding lower latency despite slightly higher MAC count than NC-SSM.

IV. ANALYSIS

Why does external SS help the NC frontend? The NC frontend's SNR estimator uses a running noise floor from the first 5 frames. At SNR < -5 dB, the noise floor estimate dominates the signal, causing the LSG to either (i) gate too aggressively (losing speech) or (ii) not gate enough (passing noise). External SS raises the effective SNR by 10–15 dB *before* the NC frontend sees the signal, shifting the operating point into the region where the SNR estimator is accurate. This is analogous to analog front-end gain staging in hardware: the SS sets the operating point, while the NC frontend performs fine-grained conditioning.

Noise propagation: SSM vs. TCN. A selective SSM computes state updates via input-dependent projections [3]:

$$u_t^{\text{SSM}} = \underbrace{\Delta(x_t)}_{\text{from } x} \cdot \underbrace{B(x_t)}_{\text{from } x} \cdot x_t. \tag{4}$$

When $x_t = s_t + n_t$ with $n_t \sim \mathcal{N}(0, \sigma_n^2 I)$, each factor contributes $O(\sigma_n^2)$ variance, yielding $\text{Var}[u_t^{\text{SSM}}] = O(\sigma_n^6)$ — *cubic* in noise power. NC-SSM mitigates this through a learned gate $\boldsymbol{\sigma}$ that blends selective dynamics toward fixed (LTI) parameters at low SNR ($\boldsymbol{\sigma} \to 0$), reducing variance to $O(\sigma_n^2)$.

In contrast, TCN uses *fixed* depthwise convolution kernels W:

$$u_t^{\text{TCN}} = (W * x)_t = W * s_t + W * n_t. \tag{5}$$

Since W is independent of x_t, noise enters *only additively*: $\text{Var}[u_t^{\text{TCN}}] = \|W\|^2 \sigma_n^2 = O(\sigma_n^2)$. This is the LTI regime

that NC-SSM's gate must *learn* to approach, whereas TCN achieves it *by construction*.

Implication. When the NC frontend has already reduced noise variability at the feature level, the backbone sees near-clean features where SSM selectivity provides diminishing returns. TCN's inherent $O(\sigma_n^2)$ noise propagation makes it naturally suited for post-frontend processing, explaining why the backbone swap incurs minimal accuracy loss (Table II).

Computational structure. SSM requires sequential state propagation: $\mathbf{h}_t = f(\mathbf{h}_{t-1}, x_t)$ over T steps, each updating $D_{\text{inner}} \times D_{\text{state}}$ elements — inherently serial. TCN's dilated convolutions decompose into L parallel depthwise Conv1d passes with $O(T \cdot D_{\text{inner}} \cdot k)$ total MACs, fully parallelizable via SIMD:

$$\text{TCN: } O(L \cdot T \cdot Dk) \text{ (parallel)}, \quad \text{SSM: } O(T \cdot D \cdot N) \text{ (serial)}, \tag{6}$$

where k=3 is kernel size, L=3 layers, D=55, N=10 state dim. On Cortex-M7, this translates to 5.4 ms (TCN) vs. 7.1 ms (SSM)—a 24% latency reduction. We verified deployment fidelity by implementing both as bare-metal C SDKs: NC-TCN achieves **100% prediction agreement** with the Python reference (logit max-diff < 0.2).

V. Conclusion

We introduced NC-TCN, demonstrating that the noise-conditional frontend—not the backbone—is the primary driver of noise robustness in sub-25K keyword spotting. Replacing the SSM scan with 3 dilated causal convolution blocks incurs <2%p accuracy loss at 0 dB while gaining full parallelism and simpler MCU deployment (verified with a bare-metal C SDK on Cortex-M7). NC-TCN-20K with external SS achieves state-of-the-art performance (96.6% clean, 89.7% at 0 dB) among sub-25K models, using only 21.7K parameters and 2.1M MACs—10.9× fewer than DS-CNN-S. Our results establish the NC frontend as a backbone-agnostic noise defense framework, and external SS as a principled operating-point bias for noise-conditional models.

References

[1] Y. Zhang, N. Suda, L. Lai, and V. Chandra, "Hello edge: Keyword spotting on microcontrollers," *arXiv preprint arXiv:1711.07128*, 2017.

[2] B.-Y. Kim, J. Kim, and J. Park, "Broadcasted residual learning for efficient keyword spotting," in *Proc. Interspeech*, 2021, pp. 4538–4542.

[3] J. H. Choi, "NC-SSM: Noise-conditioned state space models for robust on-device keyword spotting," *IEEE/ACM Trans. Audio, Speech, Language Process.*, 2026.

[4] P. Warden, "Speech commands: A dataset for limited-vocabulary speech recognition," *arXiv preprint arXiv:1804.03209*, 2018.

[5] J. Thiemann, N. Ito, and E. Vincent, "The diverse environments multi-channel acoustic noise database (DEMAND)," in *Proc. Meetings Acoust.*, vol. 19, 2013.

[6] K. J. Piczak, "ESC: Dataset for environmental sound classification," in *Proc. ACM Multimedia*, 2015, pp. 1015–1018.

[7] S. Boll, "Suppression of acoustic noise in speech using spectral subtraction," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 27, no. 2, pp. 113–120, 1979.

[8] Y. Wang, P. Getreuer, T. Hughes, R. F. Lyon, and R. A. Saurous, "Trainable frontend for robust and far-field keyword spotting," in *Proc. ICASSP*, 2017, pp. 5670–5674.

[9] S. Choi, S. Seo, B. Shin, H. Byun, M. Kersner, B. Kim, D. Kim, and S. Ha, "Temporal convolution for real-time keyword spotting on mobile devices," in *Proc. Interspeech*, 2019, pp. 3372–3376.

[10] A. Coucke, M. Chlieh, T. Gisselbrecht, D. Leez, L. Cao, and M. Lavril, "Efficient keyword spotting using dilated convolutions and gating," in *Proc. ICASSP*, 2019, pp. 6351–6355.

[11] D. S. Park, W. Chan, Y. Zhang, C.-C. Chiu, B. Zoph, E. D. Cubuk, and Q. V. Le, "SpecAugment: A simple data augmentation method for automatic speech recognition," in *Proc. Interspeech*, 2019, pp. 2613–2617.

[12] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. ICLR*, 2019.